

04 Determinants



4.4 Solving Determinants by Hand

Minors, cofactors, and Laplace expansion — computing a 3×3 determinant step by step

Linear Algebra Made Easy and Visual | Open-source code & drafts: <https://github.com/Visualize-ML>

Core Skills You Will Master

1

Minor matrix: what remains after deleting the row and column of a chosen element

2

Minor: the determinant of a minor matrix — a scalar

3

Cofactor: a sign-adjusted minor, $C_{ij} = (-1)^{i+j}M_{ij}$

4

Laplace expansion: pick any row or column, multiply each entry by its cofactor, and sum

5

Expanding along the row or column with the most zeros cuts down the work

6

Laplace expansion applies to any $n \times n$ matrix — it's the recursive foundation of the determinant

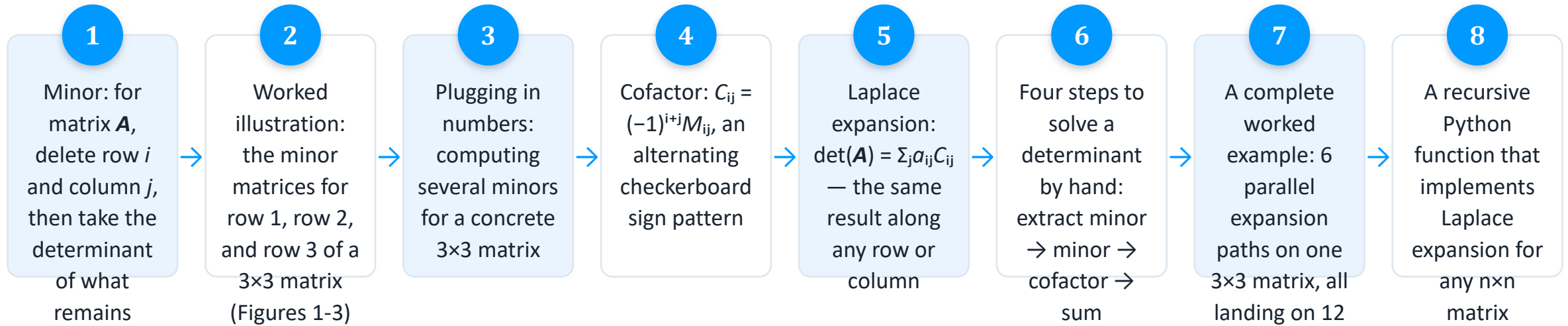
7

Implementing determinant computation with a recursive Python function sharpens algorithmic thinking

8

Working through the computation by hand builds sharper intuition for §4.3's properties (e.g. collinear rows give 0)

How This Section Unfolds



The 3×3 Formula Is Hard to Memorize

$$\det(A) = a_{11}a_{22}a_{33} + a_{12}a_{23}a_{31} + \cdots - a_{11}a_{23}a_{32}$$

- Earlier in this chapter we gave the full formula for a 3×3 determinant, but with its many terms and alternating signs, it's genuinely hard to remember
- This section introduces a more structured approach — Laplace Expansion — built on two new concepts: the minor and the cofactor
- Readers who don't need to compute determinants by hand can skip this section, or jump straight to the Python code at the end

The Minor: From Matrix to Scalar

$$M_{ij} = \det(\mathbf{A}_{ij}), \text{ where } \mathbf{A}_{ij} \text{ has shape } (n-1) \times (n-1)$$

- For an $n \times n$ matrix $\mathbf{A}$, the minor of entry a_{ij} (row i , column j) is written M_{ij}
- The minor M_{ij} is the determinant of the minor matrix $\mathbf{A}_{ij}$ — what remains of $\mathbf{A}$ after deleting row i and column j
- Note: the minor matrix $\mathbf{A}_{ij}$ is a matrix, while the minor M_{ij} is a scalar — the two should not be confused

Figure 1 · Minor Matrices for Row 1 of a 3×3 Matrix

- (a)(b)(c) show the minor matrices for a_{11} , a_{12} , a_{13} : blue is the current entry, gray is deleted, green is what remains
- $M_{11} = a_{22}a_{33} - a_{23}a_{32}$; $M_{12} = a_{21}a_{33} - a_{23}a_{31}$; $M_{13} = a_{21}a_{32} - a_{22}a_{31}$

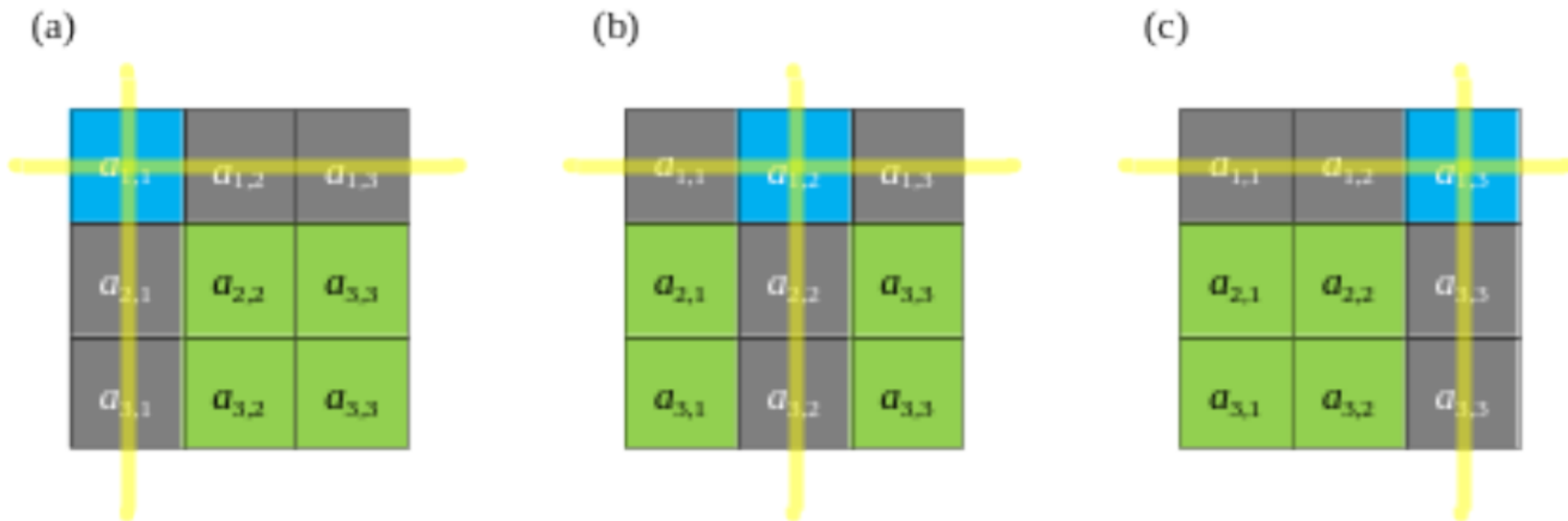


图 1. 3 × 3 方阵第 1 行各个元素的余子矩阵

Figure 2 · Minor Matrices for Row 2 of a 3×3 Matrix

- We can expand along different rows or columns — Figure 2 shows the minor matrices when expanding along row 2
- $M_{21} = a_{12}a_{33} - a_{13}a_{32}$; $M_{22} = a_{11}a_{33} - a_{13}a_{31}$; $M_{23} = a_{11}a_{32} - a_{12}a_{31}$

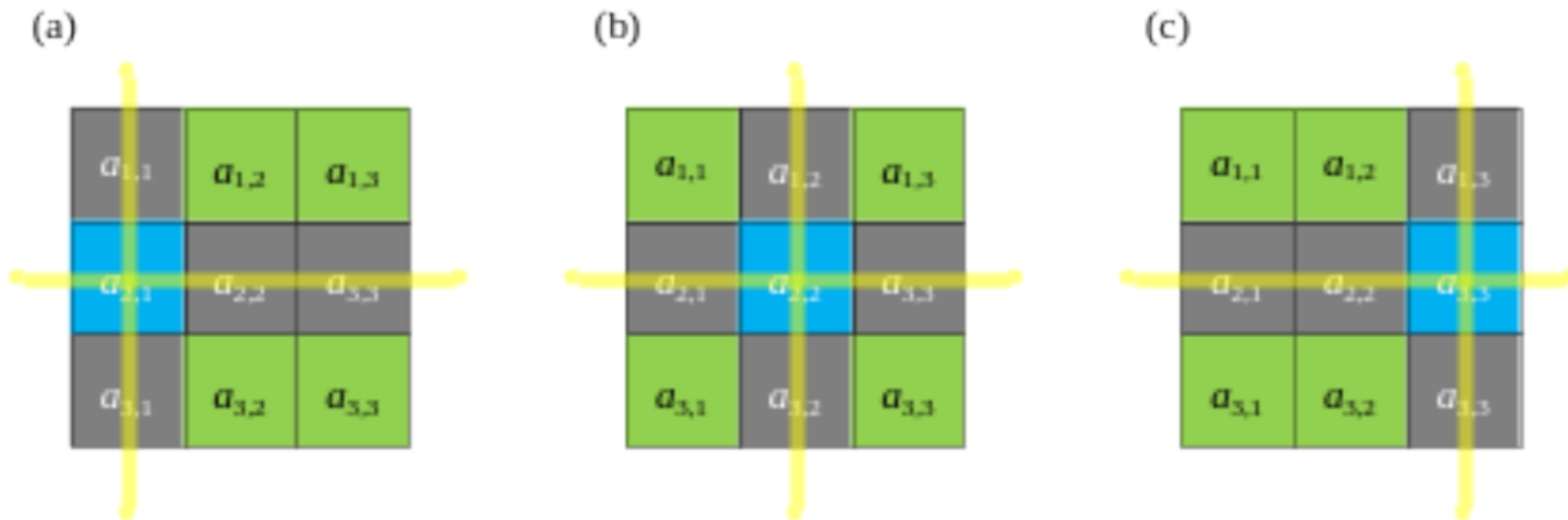


图 2. 3 × 3 方阵第 2 行各个元素的余子矩阵

Figure 3 · Minor Matrices for Row 3 of a 3×3 Matrix

- Figure 3 shows the minor matrices for row 3 — try computing these three minors yourself, following the same pattern as Figures 1 and 2
- The method is identical; only the deleted row changes, and the green cells form the minor matrix

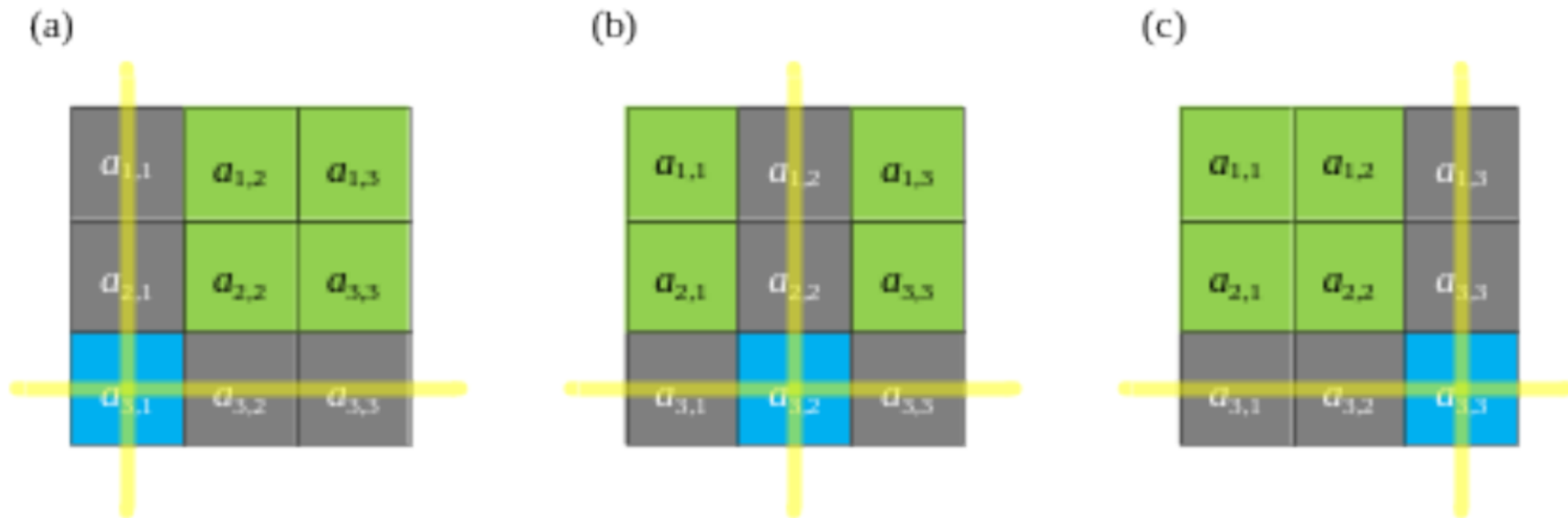


图 3. 3×3 方阵第 3 行各个元素的余子矩阵

Given a Specific 3×3 Matrix

$$A = [1 \ 2 \ 3; 4 \ 5 \ 6; 7 \ 8 \ 9]$$

- $M_{11} = |5 \ 6; 8 \ 9| = (5 \times 9) - (6 \times 8) = 45 - 48 = -3$
- $M_{12} = |4 \ 6; 7 \ 9| = (4 \times 9) - (6 \times 7) = 36 - 42 = -6$
- $M_{13} = |4 \ 5; 7 \ 8| = (4 \times 8) - (5 \times 7) = 32 - 35 = -3$
- Following the same process, try computing the remaining six minors of this matrix yourself

The Cofactor: Adding a Sign to the Minor

$$C_{ij} = (-1)^{i+j} M_{ij}$$

- A cofactor is a minor with a sign adjustment applied; $(-1)^{i+j}$ is the factor that adjusts the sign based on position
- For the matrix above: $C_{11} = (-1)^{1+1} M_{11} = 1 \times (-3) = -3$; $C_{12} = (-1)^{1+2} M_{12} = -1 \times (-6) = 6$; $C_{13} = (-1)^{1+3} M_{13} = 1 \times (-3) = -3$
- Since $(-1)^{i+j}$ forms an alternating pattern, the sign-adjustment matrix for a 3×3 matrix is $[+ \ - \ +; - \ + \ -; + \ - \ +]$

Laplace Expansion: Sum Along Any Row or Column

$$\det(\mathbf{A}) = \sum_{j=1}^n a_{ij} C_{ij}$$

- Laplace expansion offers a more systematic way to compute a determinant — you may expand along any row or any column
- Expanding the example matrix along row 1: $\det(\mathbf{A}) = a_{11}C_{11} + a_{12}C_{12} + a_{13}C_{13} = 1 \times (-3) + 2 \times 6 + 3 \times (-3) = -3 + 12 - 9 = 0$
- The three rows $[1,2,3]$, $[4,5,6]$, $[7,8,9]$ are linearly dependent (an arithmetic progression), so the determinant is exactly 0 — echoing §4.3's result that linearly dependent rows (or columns) give a zero determinant

4 Concrete Steps to Solve a Determinant by Hand

- a) Choose a row or column to expand along, and extract the corresponding minor matrices
- b) Compute the minor (a scalar) from each minor matrix
- c) Compute the cofactor of each entry — the sign-adjusted minor
- d) Apply Laplace expansion: multiply each entry in the chosen row or column by its cofactor and sum — this method generalizes to matrices of any dimension

Given Matrix A, Expand Step by Step Along Rows and Columns

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 3 & 0 & 1 \\ 1 & 2 & 1 \end{bmatrix}$$

- Following the four steps above, let's expand this specific 3×3 matrix along different rows and columns, step by step
- We'll find that no matter which row or column we choose, the final determinant comes out the same

Minor Matrices, Minors, and Cofactors for All 9 Entries

Entry	Minor Matrix $A_{i,j}$	Minor $M_{i,j}$	Cofactor $C_{i,j}$
$a_{1,1}$	$\begin{bmatrix} 0 & 1; 2 & 1 \end{bmatrix}$	-2	-2
$a_{1,2}$	$\begin{bmatrix} 3 & 1; 1 & 1 \end{bmatrix}$	2	-2
$a_{1,3}$	$\begin{bmatrix} 3 & 0; 1 & 2 \end{bmatrix}$	6	6
$a_{2,1}$	$\begin{bmatrix} 2 & 3; 2 & 1 \end{bmatrix}$	-4	4
$a_{2,2}$	$\begin{bmatrix} 1 & 3; 1 & 1 \end{bmatrix}$	-2	-2
$a_{2,3}$	$\begin{bmatrix} 1 & 2; 1 & 2 \end{bmatrix}$	0	0
$a_{3,1}$	$\begin{bmatrix} 2 & 3; 0 & 1 \end{bmatrix}$	2	2
$a_{3,2}$	$\begin{bmatrix} 1 & 3; 3 & 1 \end{bmatrix}$	-8	8
$a_{3,3}$	$\begin{bmatrix} 1 & 2; 3 & 0 \end{bmatrix}$	-6	-6

$A_{1,1}$ (delete row 1, column 1) is the matrix $\begin{bmatrix} 0 & 1; 2 & 1 \end{bmatrix}$; the other 8 minor matrices follow the same pattern — a 2×2 matrix left after deleting the corresponding row and column.

6 Parallel Paths, All Giving $\det(A) = 12$

Expansion	Laplace Expansion Formula	Substituting Values	Result
Row 1	$a_{1,1}C_{1,1}+a_{1,2}C_{1,2}+a_{1,3}C_{1,3}$	$1\times(-2)+2\times(-2)+3\times6$	12
Row 2	$a_{2,1}C_{2,1}+a_{2,2}C_{2,2}+a_{2,3}C_{2,3}$	$3\times4+0\times(-2)+1\times0$	12
Row 3	$a_{3,1}C_{3,1}+a_{3,2}C_{3,2}+a_{3,3}C_{3,3}$	$1\times2+2\times8+1\times(-6)$	12
Column 1	$a_{1,1}C_{1,1}+a_{2,1}C_{2,1}+a_{3,1}C_{3,1}$	$1\times(-2)+3\times4+1\times2$	12
Column 2	$a_{1,2}C_{1,2}+a_{2,2}C_{2,2}+a_{3,2}C_{3,2}$	$2\times(-2)+0\times(-2)+2\times8$	12
Column 3	$a_{1,3}C_{1,3}+a_{2,3}C_{2,3}+a_{3,3}C_{3,3}$	$3\times6+1\times0+1\times(-6)$	12

No matter which row or column we expand along, we arrive at $\det(A) = 12$ — exactly what the Laplace expansion theorem guarantees.

Prefer the Row or Column with the Most Zeros

$A = \begin{bmatrix} 1 & 2 & 3 \\ 3 & 0 & 1 \\ 1 & 0 & 1 \end{bmatrix} \Rightarrow \text{expanding along column 2 needs only one } 2 \times 2 \text{ determinant}$

- When applying Laplace expansion, prefer the row or column with the most zero entries — zero terms don't need to be computed, so the workload drops noticeably
- In the example above, column 2 is $[2, 0, 0]^T$; expanding along column 2, only the a_{12} term is nonzero, so only one 2×2 minor needs computing
- For diagonal, upper-triangular, or lower-triangular matrices the determinant is even simpler — it's just the product of the diagonal entries (see §4.1 and §4.2)

Implementing Laplace Expansion Recursively in Python

$$\text{determinant}(\mathbf{A}) = \sum_{col} a[0,col] \times \text{cofactor}(0, col)$$

- Code 1 shows how to compute a determinant in Python: two functions do the work — `cal_minor()` extracts the minor matrix, and `determinant()` implements Laplace expansion recursively
- The key is recursion: `determinant()` calls itself while computing each cofactor, breaking a large matrix down into smaller ones until it reaches the 2×2 or 1×1 base case

CODE 1

Computing a Determinant Recursively

LA_04_04_01.ipynb

- a) Define `cal_minor(A, row_idx, col_idx)` to extract the minor matrix
- b) Use `numpy.delete()` to drop the given row (`axis=0`), then the given column (`axis=1`), leaving the minor matrix
- c) Define `determinant(A)` to compute the determinant of matrix `A`
- d) Base cases: a 1×1 matrix returns its single entry directly; a 2×2 matrix uses the diagonal-product formula
- e) Expand along row 0: loop over each column, get the minor via `cal_minor`, recursively call `determinant()` for the cofactor, and accumulate `det += A[0, col] * cofactor` — this is the Laplace expansion step

```
## 初始化
import numpy as np

## 自定义函数, 提取余子矩阵
def cal_minor(A, row_idx, col_idx):
    A_i = np.delete(A, row_idx, axis=0)
    A_ij = np.delete(A_i, col_idx, axis=1)
    # 去掉指定的行、列
    return A_ij

## 用Laplace展开递归计算行列式
def determinant(A):
    n = A.shape[0]
    if n == 1:
        return A[0, 0]
    if n == 2:
        return A[0, 0] * A[1, 1] - A[0, 1] * A[1, 0]

    det = 0
    for col_idx in range(n):
        minor = cal_minor(A, 0, col_idx) # 沿第一行展开
        cofactor = ((-1) ** col_idx) * determinant(minor)
        # 计算代数余子式
        # 相当于 (-1) ** ((col_idx + 1) + 1) = (-1) ** col_idx
        det += A[0, col_idx] * cofactor # 计算行列式

    return det

## 测试
A = np.array([[1, 2, 3],
              [3, 0, 1],
              [1, 2, 1]])
A_det = determinant(A)
```

代码 1. 计算行列式 | LA_04_04_01.ipynb

Minor → Cofactor → Laplace Expansion

$$M_{ij} = \det(\mathbf{A}_{ij}) \Rightarrow C_{ij} = (-1)^{i+j} M_{ij} \Rightarrow \det(\mathbf{A}) = \sum_j a_{ij} C_{ij}$$

- From the minor matrix, to the minor, to the cofactor, and finally summing via Laplace expansion — these four steps form the complete workflow for solving a determinant by hand, and generalize to any $n \times n$ matrix
- This section's recursive Python implementation is the most natural way to express that workflow in code; in practice we'd typically just call `numpy.linalg.det()`, but understanding the hand-computation builds real intuition for the determinant

Key Terms in This Section

Minor Matrix $A_{i,j}$ — the $(n-1) \times (n-1)$ matrix left after deleting row i and column j

Cofactor $C_{i,j} = (-1)^{(i+j)} M_{i,j}$ — a sign-adjusted minor

Sign-Adjustment Factor $(-1)^{(i+j)}$ — forms an alternating checkerboard pattern by position

Column Expansion $\det(A) = \sum_i a_{i,j} C_{i,j}$ — fix column j , sum over row i

Sarrus' Rule A shortcut for 3×3 determinants only — see this section's Q4

Minor $M_{i,j}$ — the determinant of the minor matrix $A_{i,j}$; a scalar

Laplace Expansion Sum entries along any row or column, each times its cofactor, to get the determinant

Row Expansion $\det(A) = \sum_j a_{i,j} C_{i,j}$ — fix row i , sum over column j

Recursive Algorithm `determinant()` calls itself, reducing an $n \times n$ determinant to the 2×2 or 1×1 base case

Zero-First Heuristic Expanding along the row or column with the most zeros skips those multiplications, cutting the work

Code and Further Reading

Further Reading · `numpy.linalg.det`

<https://numpy.org/doc/stable/reference/generated/numpy.linalg.det.html> — NumPy's function for computing a determinant directly

Further Reading · `numpy.delete`

<https://numpy.org/doc/stable/reference/generated/numpy.delete.html> — the core function Code 1 uses to build the minor matrix

Further Reading · Sarrus' Rule

A quick technique for 3×3 determinants only — a special case of Laplace expansion; see Exercise Q4

Open-Source Repository

All code and figures for this book: <https://github.com/Visualize-ML>

Exercises Q1 – Q9

Q1 For each of the following three matrices, pick any row or column and compute the minors of its entries. Which row or column is most convenient to expand along? $A=[1,0,0;1,1,1;1,1,1]$; $B=[1,1,1;0,1,0;0,1,1]$; $C=[1,1,1;1,1,1;0,0,1]$.

Q3 Expand along columns to compute the determinant of $A = [1,2,3;3,0,1;1,2,1]$, and verify the result is still 12.

Q5 Modify Code 1 to add a check that the input matrix is square, and proceed with the determinant only if it is.

Q7 Write `verify_laplace(A)` that expands a 3×3 matrix along every row and every column, and checks that all 6 results agree.

Q9 Write `all_minors(A)` that returns an $n \times n$ array holding the minor of every entry of an $n \times n$ matrix A .

Q2 Use this section's Laplace expansion to compute the determinant of the following 2×2 matrix: $A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$.

Q4 Self-study the diagonal rule (Sarrus' rule), and explain how it relates to Laplace expansion.

Q6 Modify Code 1 to prefer expanding along the row or column with the most zero entries, to reduce the computation.

Q8 Use `numpy.linalg.det()` to verify that $A=[1,2,3;3,0,1;1,2,1]$ from this section's worked example has determinant 12.

Exercises Q10 – Q18

Q10 Write `all_cofactors(A)`, building on `all_minors`, that returns the matrix of all cofactors (with signs applied).

Q12 Write `laplace_expand(A, row_idx)` that expands along a chosen row, and compare its result against `numpy.linalg.det()`.

Q14 Generate 10 random 4×4 matrices; compute their determinants with a 4×4-extended recursive Laplace expansion and with `numpy.linalg.det()`, and compare results and timing.

Q16 Write `count_zeros(A)` that counts the zero entries in every row and column of a square matrix, and returns the index of the row or column with the most zeros.

Q18 Test how the runtime of recursive Laplace expansion grows as matrix size increases from 3×3 to 8×8, and plot runtime versus dimension.

Q11 Explore: what is the relationship between the transpose of the cofactor matrix and `A` itself? (Hint: self-study the adjugate matrix `adj(A)` and its link to the determinant.)

Q13 Write `laplace_expand_col(A, col_idx)` that expands along a chosen column, and compare its result against `numpy.linalg.det()`.

Q15 Explore: Code 1's determinant function only expands along the first row. Modify it to allow expansion along any chosen row.

Q17 Building on Q16, write `smart_determinant(A)` that expands along the row or column with the most zeros.

Exercises Q19 – Q26

Q19 Explore: why is Laplace expansion's time complexity $O(n!)$, while Gaussian elimination (covered later in the book) only needs $O(n^3)$?

Q21 Explore: if one row of A is a multiple of another (linearly dependent), what happens to the minors and cofactors under Laplace expansion? Verify with a concrete example that the determinant is 0.

Q23 Explore: use sympy to perform a symbolic Laplace expansion on a 3×3 matrix of symbols $[[a,b,c],[d,e,f],[g,h,i]]$, and print the fully expanded algebraic expression.

Q25 For an upper-triangular matrix, compute its determinant both with this section's Laplace expansion and with §4.2's diagonal-product formula; verify the results agree and compare the effort involved.

Q20 Write `print_all_minors(A)` that constructs and prints all 9 minor matrices of a 3×3 matrix, for teaching purposes.

Q22 Write `sarrus_3x3(A)` implementing the diagonal rule (Sarrus' rule) for 3×3 determinants, and compare its result against this section's cofactor expansion.

Q24 Write `compare_expansion_speed(A)` that counts the multiplications needed when expanding along the sparsest-in-zeros versus richest-in-zeros row or column.

Q26 Write a short summary (as code comments is fine) explaining how the minor, cofactor, and Laplace expansion build on one another.

10 Prompts to Explore Further (1/2)

1 Explain the difference between a minor and a minor matrix — why is one a scalar and the other a matrix?

2 Explain why the cofactor multiplies the minor by $(-1)^{i+j}$ — what does this sign pattern mean?

3 Explain the general Laplace expansion formula, and why expanding along any row or column gives the same determinant.

4 Explain why expanding along the row or column with the most zeros reduces the computation — what steps get skipped exactly?

5 Explain the relationship between Laplace expansion and recursion — why does computing an $n \times n$ determinant reduce to the 2×2 or 1×1 case?

10 Prompts to Explore Further (2/2)

6

Explain the base case in Code 1's determinant function — would the code still be correct without the special 2×2 case?

7

Explain why Laplace expansion has $O(n!)$ time complexity, and what that means practically for large matrices.

8

Explain the scope of the diagonal rule (Sarrus' rule) — why doesn't it generalize directly to 4×4 or higher-dimensional matrices?

9

Reflecting on this section's worked example (6 paths, all giving $\det(A)=12$), what properties of the determinant does this illustrate?

10

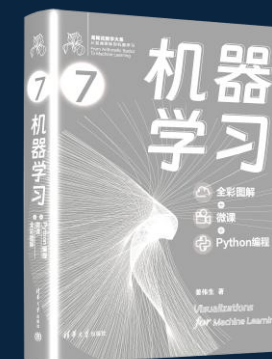
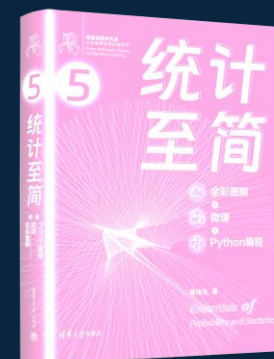
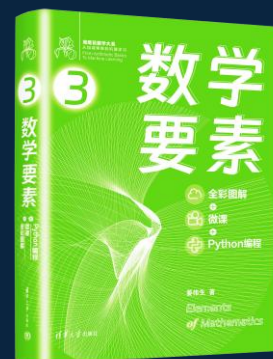
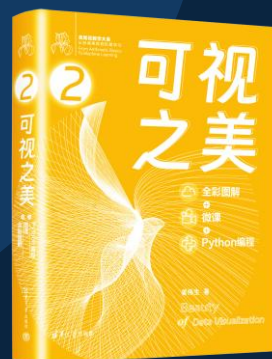
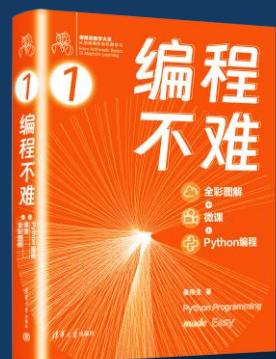
Drawing on this whole chapter (§4.1-4.4), summarize the strengths and best use cases of the geometric approach (area/volume) versus the algebraic approach (Laplace expansion) to computing determinants.

数学

数学基础

线性代数

概率统计



Python编程

数据可视化

工具

数据分析
图和网络

回归、降维
分类、聚类

实践

